OPERATING SYSTEM

PROJECT 1 – File & Process Management

1. General rule

- The project is done in groups: each group has a maximum of **3 students.**

- **The same exercises will all be scored 0 for the entire practice (even though there are scores for other exercises and practice projects).**

- Operating System: **Linux .**

2. Submission

Submit assignments directly on the course website (MOODLE), not accepting submissions via email or other forms.

Filename: **StudentID1_StudentID2_StudentID3.zip** (with StudentID1 < StudentID2< StudentID3).

Ex: Your group has 2 students: 2312001, 2312002 and 2312003, the filename is: **2312001_2312002_2312003.zip.**

Include:

- **StudentID1_StudentID2_StudentID3_Report.pdf:** Writeups should be short and sweet. Do not spend too much effort or include your source code on your writeups. The purpose of the report is to give you an opportunity to clarify your solution, any problems with your work, and to add information that may be useful in grading. If you had specific problems or issues, approaches you tried that didn't work, or concepts that were not fully implemented, then an explanation in your report may help us to assign partial credit
- **Release**: File diff (diff patch, Ex: $ git diff > **StudentID1_StudentID2_StudentID3**.patch).
- **Source**: Zip file of xv6 (the version is "made clean").

3. Demo Interviews

Your implementation is graded on completeness, correctness, programming style, thoroughness of testing, your solution, and code understanding.

When administering this course, we do our best to give a fair assessment to each individual based on each person's contribution to the project.

4. Requirements

4.1. To investigate and report

- Field `ofile[]` of `struct proc`, `file[]` of `struct ftable`.
- Trace code of syscall `open` (main code flow).
- Compare file descriptor usage in `open()`, `dup()` and `fork()`.
- Field `pid`, `state`, `context` of `struct proc`.
- Trace code of syscall `fork` (main code flow).
- Check scheduling mechanism in `scheduler()`, `sched()`, `yield()`.

4.2. To implement and report

To implement a system call, `sysinfo`, that collects below information about the running system.

- the number of opening files in whole system and list of them with their information (type, mode, reference count, list of processes reference to this open file).
- the number of processes whose state is not UNUSED, and list of them with their information (pid, state, parent's pid, number of scheduling events, list of open files).

Write user program `user/sysinfotest.c` to test the system call.
IMPORTANT: test as many cases as possitble !

Some hints:

- The system call takes one argument: a pointer to a `struct sysinfo` (add and define in `kernel/sysinfo.h`). In kernel, all required information will be filled in a `struct sysinfo` then copied back to user space.
- To declare the prototype for sysinfo() in `user/user.h` you need predeclare the existence of `struct sysinfo`

```
struct sysinfo;
int sysinfo(struct sysinfo *);
```

- The user program `user/sysinfotest.c` will invoke the `sysinfo()` system call multiple times to test as many scenarios as possible, thereby demonstrating various types of results.

5. Grade

No.	Exercise	Grade
1	Report	4
2	sysinfo	6

6. Reference

-